

ユニット 9・第15週

AI ネイティブ OS : OS 中のエージェントと LLM

📅 第15週・日程は未定 </> 演習 TD8 (解答 + 生成AIチャレンジ) ~ アニメーション 4 本

演習 TD8・解答付き (新規)

自分の micro-LLM で作るエージェント型 OS

Windows や Linux ディストリビューションは、システムの中に AI エージェントを受け入れ始めています。この演習では、コースのすべてを使ってそのアーキテクチャを小さく作ります：コース用に訓練した **micro-LLM** がスペイン語か英語の要求（理解できるのはこの2つの言語だけ）をツール呼び出しに変え、**デーモン**（`agentd`）がそれを UNIX ソケットで受け取り、**ポリシー**で検証し、破壊的なら**同意**を求め、**Landlock** で閉じ込めたプロセスで実行し、すべてを**監査ログ**に書きます。

</> ビルドとテスト

Bash

```
make -C tds                                # td8 もビルド: tinyllm、agentd、agent、
agent_exec
tds/tests/run.sh td8                       # 24 項目: モデル、要求、ポリシー、同意、Landlock、
監査
cd tds/bin/td8 && ./tinyllm -v "list the files in docs" # 要求はスペイン語か
英語のみ
```

Landlock が有効な Linux 5.13 以降が必要（`cat /sys/kernel/security/lsm`）。訓練は任意（重みは同梱）：`make -C tds/td8 train`（NumPy、CPU で約1時間）。

☑ 課題：実習レポート

この実習は短い LaTeX レポート（必要なことだけ）で提出します。扱う内容、テンプレート、例、Prism のガイドは「課題」タブにあります。生成 AI は申告すれば利用できます。

演習問題

- | | | | |
|---|--|---|-----------------|
| 1 | micro-LLM : データと訓練 | 4 | ポリシーと同意 |
| 2 | <code>mmap</code> と KV キャッシュを使う C での推論 | 5 | 多層防御 : Landlock |
| 3 | <code>agentd</code> : ソケット、スレッドプール、ID | 6 | 監査と振り返り |

TD8 ・ 1

● 解答済み・テスト済み

micro-LLM : データと訓練

課題。 `train/dataset.py` と `train/train.py` を読んでください。モデルは正確には何を学ぶのか？ なぜファイル名をランダムに生成するのか？ 手で書いた勾配が正しいとどうしてわかるのか？

解答

Python `tds/td8/train/dataset.py`

```
"""Synthetic data for the TD8 micro-LLM: requests in Spanish or English → tool call.
```

```
Every example is a text of UTF-8 bytes:
```

```
Q: lista los archivos de docs\n
A: list_files("docs")\n
```

```
The model only learns to translate the intent into ONE call from a closed catalogue of tools (the equivalent of the MCP "connectors" of an agentic OS). File and folder names are generated at random so that the model learns to COPY them from the request (attention), not to memorise them. None of this is safe by itself: the OS decides whether to run the call (policy + Landlock in agentd).
"""
```

- GPT 型の **バイト単位** の Transformer (語彙 256、コンテキスト 128、 $d = 64$ 、ヘッド 4、層 3、パラメータ約 17.5 万) を、NumPy だけ (PyTorch なし) と手書きの誤差逆伝播で訓練したもの。 `python3 train.py --check` は解析的な勾配と有限差分を比べる。

- 要求を、12 個のツールからなる閉じたカタログの**1 つ**の呼び出しに翻訳することを学ぶ。
ランダムなファイル名のおかげで、モデルは名前を覚えるのではなく、**要求から写す**
(アテンション) しかなくなる。
- **実際の結果** (`tds/td8/weights/train_log.json`) : パラメータ 174,656、12,000 ステップ
(CPU 4 スレッドで約 87 分)、新しい 500 件の要求で完全一致 **99.0%**。最も難しい
ツールは `write_note` (89%) で、引数を 2 つ写す必要があるため。重み: 698,688
バイト。
- **データの教訓**: 前の版 (v1) はテストで 99.4% だったが、演習では `viejo.txt` を
`viefe.txt` に変え、`atajo` を認識できなかった: そういう名前をほとんど
見ていなかったのだ。ネットワークを変える代わりに、データ生成器を広げた
(アルファベット全体を使い、拡張子のない名前も)。モデルの良さはデータ次第で、だから
`agentd` は決してモデルを信用しない。
- データにはわざと危険なパス (`/etc/passwd`、`../..`) を入れてある: モデルはそれを
忠実に写さなければならない。受け入れてよいかを決めるのは**モデルの仕事ではない**から。

アニメーション A9.2 で内側を見る →

TD8 ・ 2

● 解答済み・テスト済み

mmap と KV キャッシュを使う C での推論

課題。 `tinylm.c` が重みをコピーせずに読み込む方法と、新しいトークンごとにネットワークを 1
回通るだけで済む理由を説明してください。 `./tinylm -v` を実行し、その指標をユニット 4 と 6 に
結び付けてください。C と NumPy が同じものを計算することを確認してください。

解答

C tds/td8/tinylm.c

```
int tl_load(tl_model *m, const char *path)
{
    memset(m, 0, sizeof *m);
    int fd = open(path, O_RDONLY);
    if (fd == -1)
        return -1;
    struct stat st;
    if (fstat(fd, &st) == -1 || st.st_size < HEADER) {
        close(fd);
        errno = EINVAL;
    }
```

```

    return -1;
}

/* MAP_PRIVATE + PROT_READ: nobody can modify the weights through this
mapping. */
void *map = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE, fd,
0);
close(fd); /* the mapping keeps the
file alive */
if (map == MAP_FAILED)
    return -1;

const unsigned char *raw = map;
int32_t h[7];
memcpy(h, raw + 8, sizeof h);
if (memcmp(raw, "FSOLLM01", 8) != 0 || h[0] != 1 || h[4] > TL_MAX_LAYERS
|| h[3] % h[5] != 0) {
    munmap(map, (size_t)st.st_size);
    errno = EINVAL;
    return -1;
}
m->vocab = h[1]; m->ctx = h[2]; m->d = h[3]; m->n_layer = h[4]; m->n_head
= h[5]; m->d_ff = h[6];
m->map = map;
m->map_size = (size_t)st.st_size;

/* The pointers point INTO the mapping: no copy (the file is little-
endian float32). */
const float *p = (const float *)(raw + HEADER);
const int d = m->d, f = m->d_ff;
#define TAKE(dst, n) do { (dst) = p; p += (n); } while (0)
TAKE(m->wte, (long)m->vocab * d);
TAKE(m->wpe, (long)m->ctx * d);
for (int l = 0; l < m->n_layer; l++) {
    tl_layer *L = &m->layer[l];
    TAKE(L->ln1_g, d); TAKE(L->ln1_b, d);
    TAKE(L->w_qkv, (long)d * 3 * d); TAKE(L->b_qkv, 3 * d);
    TAKE(L->w_o, (long)d * d); TAKE(L->b_o, d);
    TAKE(L->ln2_g, d); TAKE(L->ln2_b, d);
    TAKE(L->w_fc, (long)d * f); TAKE(L->b_fc, f);
    TAKE(L->w_pr, (long)f * d); TAKE(L->b_pr, d);
}
TAKE(m->lnf_g, d); TAKE(m->lnf_b, d);
#undef TAKE
m->n_params = (long)(p - (const float *)(raw + HEADER));
if ((size_t)HEADER + (size_t)m->n_params * sizeof(float) != m->map_size)

```

```
{
    munmap(map, m→map_size);
    errno = EINVAL;
    return -1;
}
return 0;
}
```

実際の出力

```
$ ./tinylm -v "lista los archivos de docs"
step 0: 'l' 100.0% 'f' 0.0% 'r' 0.0%
step 1: 'i' 100.0% 'q' 0.0% '4' 0.0%
step 2: 's' 100.0% 'g' 0.0% 'q' 0.0%
step 3: 't' 100.0% 'k' 0.0% 'b' 0.0%
step 4: '_' 100.0% '-' 0.0% 'k' 0.0%
step 5: 'f' 100.0% '2' 0.0% 'c' 0.0%
step 6: 'i' 100.0% 'o' 0.0% 'n' 0.0%
step 7: 'l' 100.0% 'L' 0.0% 's' 0.0%
step 8: 'e' 100.0% 'o' 0.0% 'f' 0.0%
step 9: 's' 100.0% '(' 0.0% 'q' 0.0%
step 10: '(' 100.0% ' ' 0.0% 'g' 0.0%
step 11: '"' 100.0% ' ' 0.0% 'R' 0.0%
step 12: 'd' 100.0% 'k' 0.0% 'b' 0.0%
step 13: 'o' 100.0% 'i' 0.0% 'e' 0.0%
step 14: 'c' 100.0% 'k' 0.0% 'f' 0.0%
step 15: 's' 100.0% 'q' 0.0% 'f' 0.0%
step 16: '"' 100.0% 'z' 0.0% 'k' 0.0%
step 17: ')' 100.0% ',' 0.0% ':' 0.0%
step 18: '\n' 100.0% 'A' 0.0% ' ' 0.0%
list_files("docs")

model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)
```

実際の出力

```
$ python3 train/compare.py ./tinylm 200
200/200 respuestas idénticas entre NumPy y C
```

`PROT_READ` と `MAP_PRIVATE` を付けた `mmap` はアドレスを返し、各層のポインタはマップしたファイルの中を指す。最初のトークンが重みに触れるまで何も読めない：だからページフォールトは読み込み中ではなく、最初の推論の間に現れる。KV キャッシュは層ごと・位置ごとに、計算済みのキーと値を保存する：新しいバイトは自分の行だけを計算する。C と NumPy の推論は、新しい 200 件の要求で同じ答えを出す。

TD8 ・ 3

● 解答済み・テスト済み

agentd : ソケット、スレッドプール、ID

課題。作業領域を指定して `agentd` を起動し、`agent` で要求を送ってください（スペイン語か英語で）。コースの各ユニットから来た部分をコードの中で見つけてください。`agentd` は誰が話しかけているかをどうやって知るのでしょうか？

解答

C tds/td8/agentd.c

```
static void serve(int client)
{
    struct ucred cred = { 0 };
    socklen_t len = sizeof cred;
    if (getsockopt(client, SOL_SOCKET, SO_PEERCREDS, &cred, &len) == -1)
        return;

    char req[MAX_REQ], out[256], why[PATH_MAX + 64];
    if (read_line(client, req, sizeof req) == 0)
        return;

    /* 1. The model proposes. */
    int steps = tl_generate(&model, req, out, sizeof out, NULL, NULL);
    if (steps < 0) {
        send_str(client, "error: request too long\n");
        audit(cred.uid, cred.pid, req, "-", "deny", "too long");
        return;
    }
    send_fmt(client, "model: %s\n", out);

    /* 2. The operating system disposes. */
    call_t c;
    if (parse_call(out, &c) == -1) {
```

```

        send_str(client, "policy: deny (the model produced an invalid
call)\n");
        audit(cred.uid, cred.pid, req, out, "deny", "invalid call");
        return;
    }
    decision_t d = policy(&c, why, sizeof why);
    send_fmt(client, "policy: %s (%s)\n", DECISION[d], why);
    if (d == DENY) {
        audit(cred.uid, cred.pid, req, out, "deny", why);
        return;
    }
    if (d == ASK) {
        char answer[16];
        send_fmt(client, "ask: allow the agent to run %s? [y/N]\n", out);
        read_line(client, answer, sizeof answer);
        if (answer[0] != 'y' && answer[0] != 'Y') {
            send_str(client, "cancelled by the user\n");
            audit(cred.uid, cred.pid, req, out, "ask", "refused by the
user");
            return;
        }
    }
}

/* 3. Sandboxed execution. */
const char *outcome = run_tool(client, &c);
audit(cred.uid, cred.pid, req, out, d == ASK ? "ask:approved" : "allow",
outcome);
}

```

実際の出力

```

$ ./agent "lista los archivos de docs"
model: list_files("docs")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
2 entries in docs
-      0  plan.md
-      0  syllabus.pdf

```

実際の出力

```
$ ./agent "muéstrame notas.txt"
model: read_file("notas.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Laboratorio TD8
fork, exec, pipe, mmap, sockets
```

実際の出力

```
$ ./agent "busca 'error' en log.txt"
model: search("error", "log.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
log.txt:2: error: disk full
log.txt:4: error: timeout
2 matching line(s) for "error"
```

実際の出力

```
$ ./agent "¿qué versión del núcleo tengo?"
model: system_info()
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Linux 7.0.0-34-generic (x86_64), 28 CPUs
memory: 32.8 GB total, 6.9 GB free; 3110 processes; up 4 h 7 min
```

実際の出力

```
$ for i in $(seq 8); do ./agent "lista mis archivos" & done; wait
8
```

AGENTD の部品	ユニット
権限 0600 の UNIX ソケット	U8・U3
<code>SO_PEERCRE</code> : カーネルが教えるクライアントの uid と pid	U8
有限のキューを持つスレッドプール (セマフォ + ミューテックス)	U7
<code>mmap</code> で共有する重み	U6
実行器の <code>fork</code> + <code>execv</code> 、出力はパイプ経由	U4・U5
停止は <code>SIGTERM</code> 、 <code>SIGPIPE</code> は無視	U6

TD8 ・ 4

● 解答済み・テスト済み

ポリシーと同意

課題。エージェントに、作業領域のファイルの削除 (「はい」と「いいえ」の両方で答える)、`/etc/passwd` の削除、`../../etc/shadow` の読み込み、そして作業領域内にある `/etc/passwd` へのシンボリックリンク `atajo` の読み込みを頼ってください。それぞれの場合に決めるのは誰か：モデルか OS か？

解答



C tds/td8/agentd.c

```
static int inside_workspace(const char *rel, char *why, size_t whysz)
{
    if (rel[0] == '/' || rel[0] == '~') {
        snprintf(why, whysz, "absolute path outside the workspace");
        return 0;
    }
    if (strcmp(rel, "..") == 0 || strncmp(rel, "../", 3) == 0 || strstr(rel,
"/../") || strstr(rel, "/..") ) {
        snprintf(why, whysz, "'..' leaves the workspace");
        return 0;
    }
    /* Resolve symbolic links: the file (or, if it does not exist yet, its
directory). */
    char full[PATH_MAX * 2], real[PATH_MAX];
    snprintf(full, sizeof full, "%s/%s", workspace, rel);
    if (realpath(full, real) == NULL) {
        char *slash = strrchr(full, '/');
        *slash = '\0';
        if (realpath(full, real) == NULL) {
            snprintf(why, whysz, "directory does not exist");
            return 0;
        }
    }
    size_t n = strlen(workspace);
    if (strncmp(real, workspace, n) != 0 || (real[n] != '\0' && real[n] !=
'/')) {
        snprintf(why, whysz, "resolves to %s, outside the workspace", real);
        return 0;
    }
    return 1;
}

static decision_t policy(const call_t *c, char *why, size_t whysz)
{
    const tool_t *t = c->tool;
    if (strcmp(t->name, "none") == 0) {
        snprintf(why, whysz, "no tool for this request");
        return DENY;
    }
    if (policy_on && t->path_arg >= 0 && !inside_workspace(c->args[t
->path_arg], why, whysz))
        return DENY;
    if (t->consent) {
```

```
    snprintf(why, whysz, "destructive action: needs the user's consent");
    return ASK;
}
    snprintf(why, whysz, policy_on ? "tool in the catalogue, path inside the
workspace" : "POLICY DISABLED");
    return ALLOW;
}
```

実際の出力

```
$ ./agent -n "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> n (automatic answer)
cancelled by the user
```

実際の出力

```
$ ./agent -y "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> y (automatic answer)
[sandbox: Landlock ABI 8, workspace only]
viejo.txt deleted
```

実際の出力

```
$ ./agent "borra /etc/passwd"
model: delete_file("/etc/passwd")
policy: deny (absolute path outside the workspace)
```

実際の出力

```
$ ./agent "lee ../../etc/shadow"
model: read_file("../../etc/shadow")
policy: deny ('..' leaves the workspace)
```

実際の出力

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: deny (resolves to /etc/passwd, outside the workspace)
```

実際の出力

```
$ ./agent "cuéntame un chiste"
model: none()
policy: deny (no tool for this request)
```

モデルは頼まれたものを常に提案する（`delete_file("/etc/passwd")` も含めて）。
決めるのは `agentd` : 絶対パス、`..`、外へ抜け出すリンク（`realpath` で解決）は拒否し、
破壊的なものはユーザに尋ねる。

アニメーション A9.4 で自分の要求を試す（スペイン語か英語で）→

TD8 ・ 5

● 解答済み・テスト済み

多層防御：Landlock

課題。 `--no-policy` を付けて `agentd` を再起動し（ポリシーのバグをまねる）、攻撃を繰り返してください。次に `agent_exec` を `--no-sandbox` ありとなしで直接実行してください。今度はどの層があなたを止めますか？

解答

C tds/td8/agent_exec.c

```
static int sandbox(const char *workspace)
{
    int abi = ll_create(NULL, 0, LANDLOCK_CREATE_RULESET_VERSION);
    if (abi < 1)
        return 0;

    __u64 fs = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_WRITE_FILE |
LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR | LANDLOCK_ACCESS_FS_REMOVE_DIR |
LANDLOCK_ACCESS_FS_REMOVE_FILE |
```

```

        LANDLOCK_ACCESS_FS_MAKE_CHAR | LANDLOCK_ACCESS_FS_MAKE_DIR |
LANDLOCK_ACCESS_FS_MAKE_REG |
        LANDLOCK_ACCESS_FS_MAKE_SOCKET | LANDLOCK_ACCESS_FS_MAKE_FIFO |
LANDLOCK_ACCESS_FS_MAKE_BLOCK |
        LANDLOCK_ACCESS_FS_MAKE_SYM;
    if (abi ≥ 2) fs |= LANDLOCK_ACCESS_FS_REFER;
    if (abi ≥ 3) fs |= LANDLOCK_ACCESS_FS_TRUNCATE;
    if (abi ≥ 5) fs |= LANDLOCK_ACCESS_FS_IOCTL_DEV;

    struct landlock_ruleset_attr attr = { .handled_access_fs = fs };
    size_t size = sizeof(__u64); /* ABI 1-3:
only the fs field */
    if (abi ≥ 4) {
        attr.handled_access_net = LANDLOCK_ACCESS_NET_BIND_TCP |
LANDLOCK_ACCESS_NET_CONNECT_TCP;
        size = offsetof(struct landlock_ruleset_attr, handled_access_net) +
sizeof(__u64);
    }
    if (abi ≥ 6) {
        attr.scoped = LANDLOCK_SCOPE_ABSTRACT_UNIX_SOCKET |
LANDLOCK_SCOPE_SIGNAL;
        size = sizeof attr;
    }
    int rs = ll_create(&attr, size, 0);
    exit_if(rs == -1, "landlock_create_ruleset");

    /* Everything inside the workspace (minus executing programs), read-only
/proc. */
    exit_if(ll_add_path(rs, workspace, fs & ~
(__u64)LANDLOCK_ACCESS_FS_EXECUTE) == -1, "landlock_add_rule workspace");
    exit_if(ll_add_path(rs, "/proc", LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR) == -1,
"landlock_add_rule /proc");

    exit_if(prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1, "prctl");
    exit_if(syscall(SYS_landlock_restrict_self, rs, 0) == -1,
"landlock_restrict_self");
    close(rs);
    return abi;
}

static void limits(void)
{
    struct rlimit cpu = { 2, 3 }, fsize = { 1 << 20, 1 << 20 }, nofile = {
64, 64 };

```

```
exit_if(setrlimit(RLIMIT_CPU, &cpu) == -1, "setrlimit cpu"); /*
SIGXCPU after 2 s */
exit_if(setrlimit(RLIMIT_FSIZE, &fsize) == -1, "setrlimit fsize"); /*
SIGXFSZ past 1 MiB */
exit_if(setrlimit(RLIMIT_NOFILE, &nofile) == -1, "setrlimit nofile");
alarm(5); /*
SIGALRM: real time */
}
```

実際の出力

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: atajo: Permission denied
```

実際の出力

```
$ ./agent "lee /etc/passwd"
model: read_file("/etc/passwd")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/passwd: Permission denied
```

実際の出力

```
$ ./agent_exec workspace read_file /etc/hostname; ./agent_exec --no-sandbox
workspace read_file /etc/hostname
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/hostname: Permission denied
[sandbox: DISABLED]
host
```

何かに触れる前に、`agent_exec` は自分を閉じ込める：CPU、ファイルサイズ、ディスクリプタの上限（`setrlimit`）、5 秒のアラーム、`no_new_privs`、そして作業領域（と読み取り専用の `/proc`）だけを許し、TCP 接続とサンドボックス外へのシグナルを拒否する Landlock の規則。ポリシーを無効にしても、カーネルが `Permission denied` と答える。同じプログラムでも、サンドボックスなしならファイルを読めてしまう。

TD8 ・ 6

● 解答済み・テスト済み

監査と振り返り

課題。 `audit.log` を読んでください。エージェントが何を、誰の代わりに行ったかを再構成できるのはどんな情報ですか？ このエージェント型 OS を実際に使うには何が足りませんか？

解答



実際の出力

```
2026-09-30T13:14:12 uid=1000 pid=364726 request="lista los archivos de docs"
call=list_files("docs") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364735 request="muéstrame notas.txt"
call=read_file("notas.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364744 request="¿cuántas líneas tiene
main.c?" call=count_lines("main.c") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364753 request="busca 'error' en log.txt"
call=search("error", "log.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364762 request="anota 'examen el lunes' en
agenda.md" call=write_note("agenda.md", "examen el lunes") decision=allow
outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364771 request="crea la carpeta proyectos"
call=make_dir("proyectos") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364780 request="¿cuánto espacio libre
queda?" call=disk_usage() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364789 request="¿qué versión del núcleo
tengo?" call=system_info() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364798 request="show running processes"
call=processes() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364807 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask outcome=refused by the user
2026-09-30T13:14:13 uid=1000 pid=364815 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask:approved outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364824 request="borra /etc/passwd"
call=delete_file("/etc/passwd") decision=deny outcome=absolute path outside
the workspace
2026-09-30T13:14:13 uid=1000 pid=364832 request="lee ../etc/shadow"
call=read_file("../etc/shadow") decision=deny outcome='..' leaves the
workspace
2026-09-30T13:14:13 uid=1000 pid=364840 request="lee atajo"
call=read_file("atajo") decision=deny outcome=resolves to /etc/passwd,
outside the workspace
2026-09-30T13:14:13 uid=1000 pid=364848 request="cuéntame un chiste"
call=none() decision=deny outcome=no tool for this request
2026-09-30T13:14:13 uid=1000 pid=364873 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364875 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364877 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364876 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364880 request="lista mis archivos"
```

```
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364879 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364881 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364878 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
```

- 各行には日付、クライアントの uid と pid（クライアントではなくカーネルが教えるもの）、要求、提案された呼び出し、判断、結果が入っている：Windows がエージェントのアカウントで求めるように、エージェントが行ったことをユーザが頼んだことと区別できる。
- 足りないものはたとえば：エージェント専用のユーザ（今は教員として動いている）、`seccomp` フィルタ、`cgroups` によるメモリ上限、エージェント自身からのログの保護、そしてエージェントがファイルの中身を読むようになったときのプロンプトインジェクション対策。

チェックリスト

生成AIチャレンジ・ エージェントの新しいツール...そして攻撃



目標。 エージェント型 OS に `copy_file(source, destination)` ツールを加える：データを生成し、micro-LLM を再訓練し、ポリシーと実行器を拡張する。次に、エージェントに作業領域の外へファイルをコピーさせようとして、どの層が止めるかを確かめる。

最初のプロンプト。 好きなツール（Claude、ChatGPT、Gemini、Copilot、Codex...）で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

C で書いた小さなエージェント型 OS を扱っています：micro-LLM (tinyllm.c) が要求をツール呼び出しに変え、agentd がポリシーを適用し、agent_exec が Landlock のサンドボックス内でツールを実行します [dataset.py、agentd.c の TOOLS 表、agent_exec.c の main() を貼り付け]。copy_file(source, destination) を加えたいです。次の手順を導いてください：(1) dataset.py にスペイン語と英語のテンプレートを加える（micro-LLM が理解するのはこの 2 つの言語だけ）。(2) 再訓練し、前後の精度を測る。(3) 引数が「両方とも」作業領域内にとどまるようポリシーを拡張する。(4) agent_exec にコピーを実装する。その後、悪意のある要求を 5 つ（絶対パス、'..'、シンボリックリンク、テキスト内のインジェクション）考え、それぞれをどの層（パーサ、ポリシー、Landlock）が止めるかを予想するのを手伝ってください。

結果の確かめ方

- 再訓練したモデルの、既存ツールでの精度が 2 ポイントより多く下がらない（`train.py --eval`）。
- 作業領域外を宛先にした `copy_file` は `policy: deny` で終わり、`--no-policy` なら `Permission denied` (Landlock) で終わる。
- `audit.log` に 5 回の試みがその判断とともに記録される。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。